

CS 334 — Homework 2 Extra Credit

Nathaniel Schrader — CS 334 Section 2 — Fall 2026

Honor Code Statement:

THIS HOMEWORK IS MY OWN WORK, WRITTEN WITHOUT COPYING FROM OTHER STUDENTS OR DIRECTLY FROM LARGE LANGUAGE MODELS SUCH AS CHATGPT.

Any collaboration or external resources have been properly acknowledged.

Nathaniel Schrader

Q4 — Weighted Linear Regression

(a) When some data points are more reliable than others. For example, if some measurements come from a precise instrument and others from a cheap, noisy one, you'd want the precise ones to count more. Another example is weighting recent data more heavily in a time series.

(b) Let W be the $N \times N$ diagonal matrix with $W_{ii} = w^{(i)}$, and let $\vec{r} = X\vec{\theta} - \vec{y}$, so $r_i = \vec{\theta} \cdot \vec{x}^{(i)} - y^{(i)}$. Since W is diagonal,

$$(X\vec{\theta} - \vec{y})^T W (X\vec{\theta} - \vec{y}) = \vec{r}^T W \vec{r} = \sum_{i=1}^N w^{(i)} r_i^2 = \sum_{i=1}^N w^{(i)} (\vec{\theta} \cdot \vec{x}^{(i)} - y^{(i)})^2,$$

and $\lambda \|\vec{\theta}\|_2^2 = \lambda \vec{\theta}^T \vec{\theta}$. So $J(\vec{\theta}) = (X\vec{\theta} - \vec{y})^T W (X\vec{\theta} - \vec{y}) + \lambda \vec{\theta}^T \vec{\theta}$.

(c) Expanding (using $W = W^T$):

$$J(\vec{\theta}) = \vec{\theta}^T X^T W X \vec{\theta} - 2\vec{y}^T W X \vec{\theta} + \vec{y}^T W \vec{y} + \lambda \vec{\theta}^T \vec{\theta}$$

Take the gradient and set it to zero:

$$\nabla J = 2X^T W X \vec{\theta} - 2X^T W \vec{y} + 2\lambda \vec{\theta} = 0$$

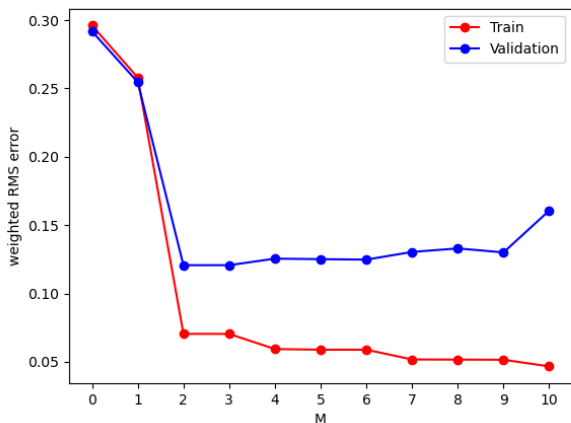
$$(X^T W X + \lambda I) \vec{\theta} = X^T W \vec{y}$$

$$\vec{\theta}^* = (X^T W X + \lambda I)^{-1} X^T W \vec{y}$$

(d) From `linear_regression.py`:

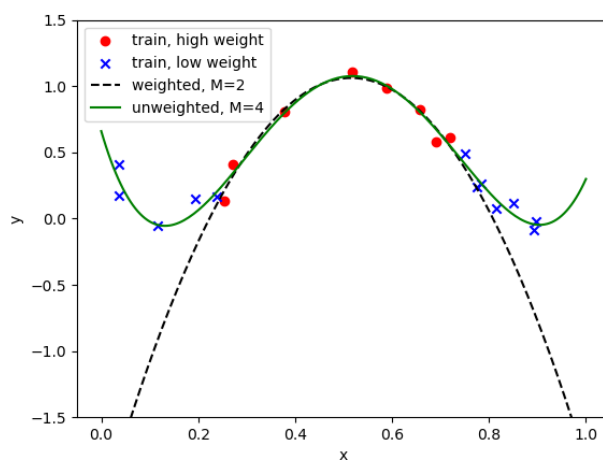
```
1 def weighted_ls_closed_form_solution(X, y, weights, reg_param=0):
2     d = X.shape[1]
3     W = np.diag(weights)
4     # theta = (X^T W X + lambda I)^-1 X^T W y
5     A = np.dot(np.dot(X.T, W), X) + reg_param * np.eye(d)
6     theta = np.dot(np.dot(np.dot(np.linalg.pinv(A), X.T), W), y)
7     return theta
```

(e) Same sweep as Q3.2(b) but with the weighted fit. Since the fit minimizes weighted error, I measured error with weighted RMS, $\sqrt{\sum_i w^{(i)} r_i^2 / \sum_i w^{(i)}}$.



M	weighted RMS		unweighted RMS	
	train	val	train	val
0	0.2960	0.2917	0.465	0.421
1	0.2579	0.2547	0.529	0.472
2	0.0705	0.1207	0.731	0.486
3	0.0704	0.1207	0.719	0.490
4	0.0594	0.1255	0.238	0.232
5	0.0589	0.1251	0.139	0.133
6	0.0589	0.1248	0.144	0.143
7	0.0517	0.1304	0.462	1.021
8	0.0516	0.1330	0.452	1.249
9	0.0515	0.1301	0.462	0.985
10	0.0467	0.1606	0.275	2.383

No, the answer changes to $M = 2$ (weighted validation RMS 0.1207, basically tied with $M = 3$, so I'd pick the simpler one).



The weights are either 1.001 or 0.001, so the fit basically only sees the high-weight points. Those are all in the middle of the curve (x from about 0.25 to 0.72), and a parabola fits that part well. The ends of the curve, which needed $M = 4$ before, barely count anymore. Higher degrees just fit noise in the high-weight points. The plot shows this: the weighted $M = 2$ fit matches the red points but goes way off at the ends.